UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Concurrent Server TCP/IP

Concurrent Real Time Programming

Furlan Davide

July 20, 2026

Abstract

- **Objective:** Implementation of a real-time executive capable of dynamic task activation via a distributed TCP/IP interface.
- **Core Problem:** Ensuring system determinism when task sets change at runtime.
- **Solution:** A multi-threaded server utilizing **Rate Monotonic Scheduling (RMS)** principles and **Response Time Analysis (RTA)** to validate schedulability before task admission.
- **Key Achievement:** A robust synchronization mechanism managing the tension between asynchronous network commands and synchronous periodic task execution.

System Requirements & Constraints

Task Model

Each task T_i is defined by a triplet (C_i, P_i, D_i):

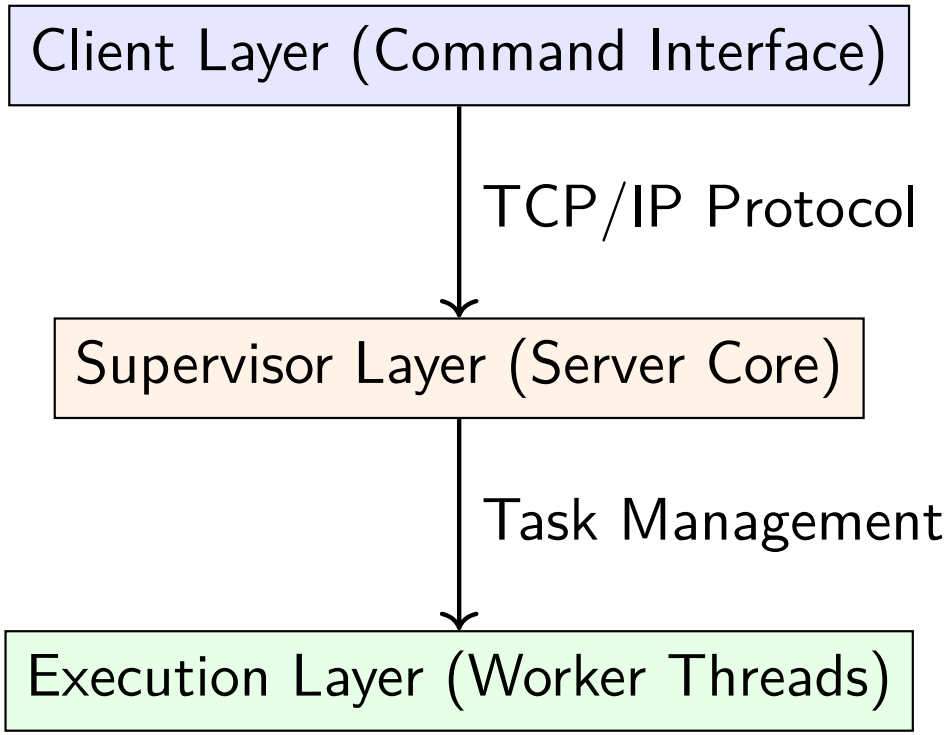
- C_i: Computation time (CPU usage).
- P_i: Periodicity.
- D_i: Deadline ($D_i \leq P_i$).

Operational Constraints

- **Dynamic Admission:** Tasks are activated/deactivated via remote TCP commands.
- **Resource Limits:** A fixed capacity for active tasks ($N = 4$) and thread limits for client connections.
- **Determinism:** Use of monotonic clocks to prevent drift and jitter.

High-Level Architecture

The system follows a three-tier hierarchical architecture:



High-Level Architecture 2

- **Client Layer:** Parses user input and transmits encoded command payloads.
- **Supervisor Layer:** Manages the connection pool, handles command parsing, and performs schedulability checks.
- **Execution Layer:** Dedicated threads for periodic task routines using absolute time-based sleeping.

Data Structures & Task Modeling

To separate static definitions from runtime instances, we implement a dual-structure approach:

Static: `Task`

- `id`
- `cpu_usage` (C_i)
- `period` (P_i)
- `deadline` (D_i)
- `routine` (Function Pointer)

Dynamic: `ActiveTask`

- `task_id` (Link to catalog)
- `instance_id` (For instance tracking)
- `thread_id` (Handle for lifecycle)
- `client_port` (Ownership tracking)

The `TASK_CATALOG` acts as the "Read-Only" truth, while `tasks_active` represents the "Mutable" state of the system.

Schedulability Policy: The Decision Engine

Before admitting a new task T_{new}, the system performs a hierarchical feasibility test:

1. **Utilization Factor Test (Necessary Condition):** Calculate total utilization U:

$$U = \sum_{i=1}^{n} \frac{C_i}{P_i} \leq 1.0$$

   If $U > 1.0$, the task is rejected immediately.
2. **Sufficient Condition Test (Liu & Layland):** If $U \leq n(2^{1/n} - 1)$, the system is guaranteed to be schedulable. In our implementation, we use the lower bound of $\ln(2) \approx 0.693$ for rapid verification.
3. **Response Time Analysis (Exact Test):** If $0.693 < U \leq 1.0$, we proceed to the iterative RTA to determine if a deadline miss is possible.

Mathematical Logic: RTA Algorithm

The algorithm computes the worst-case response time R_i for each task in the set.

The Iterative Equation

For each task i, we solve for R_i using:

$$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{P_j} \right\rceil C_j$$

where $hp(i)$ is the set of higher-priority tasks.

Implementation in `rta.c`:

- **Sorting:** Tasks are sorted by period P_i.
- **Convergence:** The loop continues until $R_i^{(k)} = R_i^{(k-1)}$.
- **Failure Condition:** If $R_i > D_i$ the task is **unschedulable**.

Concurrency Model: Threading Hierarchy

How do we maintain the period of a task?

- **Connection Management:** Each client connection is handled by a `connectionHandler` thread. This prevents a single slow client from blocking the server's ability to accept new connections.
- **Task Execution:** Once a task is admitted, a worker thread is spawned (`handling_active_task`).
  - The thread executes the `routine()`.
  - It uses `clock_nanosleep(CLOCK_MONOTONIC, ...)` to ensure precise periodic execution.
  - It monitors the `active` flag to allow for graceful deactivation.

Synchronization and Resource Management

To prevent race conditions in a multi-threaded, multi-client environment, we employed:

Mutexes:
- `active_tasks_mutex`: Protects task array and RTA.
- `mutex`: Synchronizes client connection buffer.
- `log_mutex`: Ensures atomic, clean logging output.

Semaphores: `free_slots_sem` limits concurrent active tasks.

Condition Vars: `roomAvailable` blocks new incoming connections when the client buffer is full.

Implementation: Deterministic Timing Control

A critical component of a real-time executive is the ability to handle "next wake-up" calculations without cumulative error.

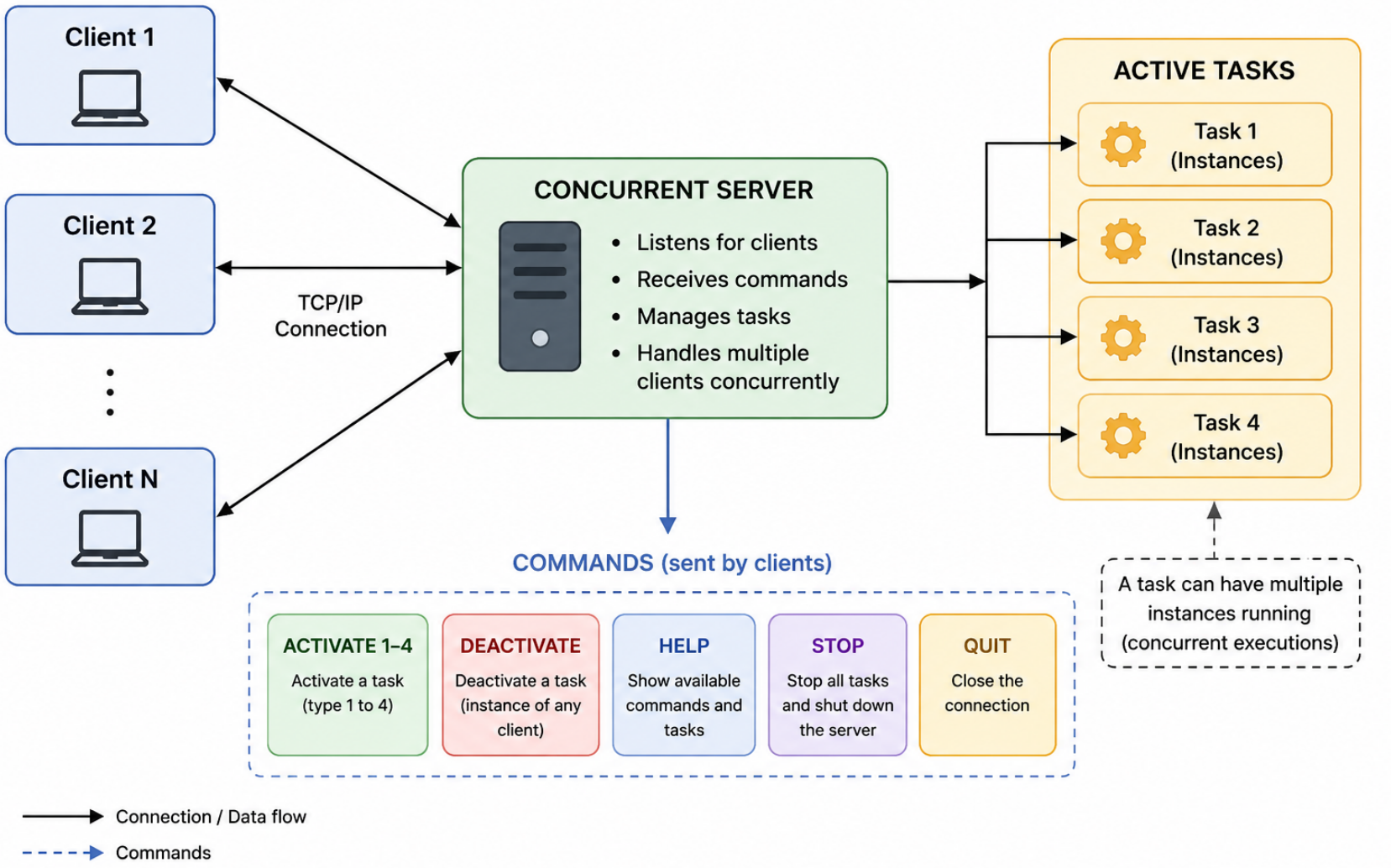
Absolute Time Calculation

Instead of relative sleeps (which accumulate drift), the system calculates the **Absolute Wake-up Time**:

1. Fetch current monotonic time.
2. Calculate $T_{next} = T_{current} + P_i$.
3. Use `clock_nanosleep` with `TIMER_ABSTIME`.

Handling Nanosecond Overflows: The code manually adjusts `tv_sec` and `tv_nsec` to handle the overflow where $nsec \geq 10^9$, ensuring the kernel receives a valid `timespec` structure.

Concurrent Client-Server Architecture



Conclusion & Future Work

- **Future Improvements:**
  - Compare Rate Monotonic with alternative fixed-priority policies.
  - Extend evaluation to dynamic-priority algorithms (e.g., EDF).
  - Evaluate system performance under high CPU utilization and overloaded conditions.